

一阶逻辑推理

祝恩
计算机学院
国防科技大学

学习内容

1. 命题推理与一阶推理

量词实例化推理规则；退化到命题推理

2. 一阶推理规则与合一

一阶假言推理规则；合一代换

3. 前向链接(使用假言推理规则)

一阶限定子句；前向链接；高效的前向链接

4. 归结

一阶逻辑的CNF；归结推理规则；归结证明实例；归结策略

全称量词的推理规则;
存在量词的推理规则;
退化到命题推理。

1 命题推理与一阶推理



代换(Substitution)

- FOL is Similar to PL
 - Important differences
 - Quantifiers
 - Variables
- Important concept: *substitution*(代换)
 - $\text{Subst}(\theta, \alpha)$, θ is like $\{x/\text{Michael}, y/\text{Bob}\}$
 - replace variables with terms

$$(\forall x)(\text{Man}(x) \Rightarrow \text{Mortal}(x))$$

$$\text{Subst}(\{x/\text{Michael}\}, (\forall x)(\text{Man}(x) \Rightarrow \text{Mortal}(x)))$$

$$\text{Man}(\text{Michael}) \Rightarrow \text{Mortal}(\text{Michael})$$

全称量词实例化推理规则

- 全称量词实例化推理规则（UI规则）
- 全称量化的语句可以推理出用没有变量的项（基项）代换全称量化变量得到的语句。

$$\frac{\forall v \quad \alpha}{\text{SUBST}(\{v / g\}, \alpha)}$$

$$\frac{\forall x \quad \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)}{\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})}$$

存在量词实例化推理规则

- 存在量词实例化推理规则（EI规则）
- 存在量化的语句可以推理出
skolem常量代换存在量化变量得到的语句

$$\frac{\exists v \quad \alpha}{\text{SUBST}(\{v / k\}, \alpha)}$$

k is a *Skolem* constant

在全称量词辖域内的存在量词

□ Instantiate

$$(\forall x_1) \dots (\forall x_{k-1}) (\exists x_k) (Qx_{k+1}) \dots (Qx_n) M(x_1, \dots, x_k, \dots, x_n)$$

replace x_k with a Skolem function $f(x_1, \dots, x_{k-1})$

$$(\forall x_1) \dots (\forall x_{k-1}) (Qx_{k+1}) \dots (Qx_n) M(x_1, \dots, f(x_1, \dots, x_{k-1}), \dots, x_n)$$

一阶语句命题化

- 存在量化语句可以被实例化语句代替
- 全称量化语句可以被所有可能的实例化集代替

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 $\text{King}(\text{John})$
 $\text{Greedy}(\text{John})$
 $\text{Brother}(\text{Richard}, \text{John})$

实例化后删除全称量化语句

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$
 $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$
 $\text{King}(\text{John})$
 $\text{Greedy}(\text{John})$
 $\text{Brother}(\text{Richard}, \text{John})$

King(John)等可看作命题符号，
该知识库本质上就是命题逻辑

知识库中包含函词时

- 当知识库中包含函词时，**基项置换集**可能是无限的！例如，如果知识库包括符号 *Father*，那么**可以构造无限多个嵌套项**，如 *Father(Father(Father(John)))*。我们的命题算法处理无限的语句集合时有困难。

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
King(John)
Greedy(John)
Greedy(Father(John))

- Jacques Herbrand (1930) 针对这种情况提出了一个著名的定理，这就是**如果某个语句被原始的一阶知识库蕴涵**，则存在一个**只涉及命题化知识库的有限子集**的证明。

半可判定性

- 如果语句不被蕴涵，我们能否做出判断？对于一阶逻辑，答案是否定的。证明程序可以不断进行下去，生成越来越深的**嵌套项**，我们不知道它是陷在无望的循环中，还是快要证明出结果了。这很像图灵机的**停机问题**。Alan Turing (1936) 和Alonzo Church (1936) 用不同的方法证明了这种事件状态的必然性。一阶逻辑的蕴涵问题是**半可判定的**——存在算法能够证明被蕴涵的语句，不存在算法否定不被蕴涵的语句。

一阶假言推理规则

合一代换

标准化分离

最一般合一代换

2 一阶推理规则与合一



假言推理规则

□ 证明结论Evil(John)

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

King(John)
Greedy(John)
Brother(Richard, John)

两步，运用两条推理规则：

1. 全称量词实例化推理规则
2. (命题逻辑的)假言推理规则

一般化的假言推理规则

- 一般化的假言推理规则将假言推理规则从命题逻辑提升到一阶逻辑

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

- 代换 θ 使得
 $\text{SUBST}(\theta, p_1' \wedge \dots \wedge p_n') = \text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n)$

找到使不同的逻辑表示变得相同的代换

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 $\text{King}(\text{John})$
 $\text{Greedy}(\text{John})$
 $\text{Brother}(\text{Richard}, \text{John})$

合一代换

- 找到使不同的逻辑表示变得相同的代换的过程称为**合一**。**合一算法**Unify以两条语句为输入，如果存在合一代换则返回该**合一代换（合一元）**，否则返回失败。

$$\text{UNIFY}(p, q) = \theta, \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x / \text{Jane}\}$$

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Bill})) = \{x / \text{Bill}, y / \text{John}\}$$

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Mother}(y))) = \{y / \text{John}, x / \text{Mother}(\text{John})\}$$

标准化分离

- Consider the sentence
 - $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Elizabeth}))$
= Fail
 - This fails because x cannot take on two values
 - But “Everyone knows Elizabeth” and it should not fail
 - Must **standardize** apart one of the two sentences to eliminate reuse of variable ()
 - $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x_1, \text{Elizabeth}))$

最一般合一代换MGU

□ 合一代换可能不唯一

- UNIFY (Knows(John,x), Knows(y,z))
- $\{y/\text{John}, x/z\}$ or $\{x/\text{John}, y/\text{John}, z/\text{John}\}$
- Knows (John, z) or Knows (John, John)
- First unifier is more **general** than second because it places fewer restrictions on the values of the variables

□ There is a *most general unifier (MGU)* for every unifiable pair of expressions

一阶限定子句
简单的前向链接
高效的前向链接

3 前向链接



命题逻辑的限定子句与前向链接

- **限定子句**：正好只有一个正文字的子句。
 - 它可以是原子语句；
 - 或是蕴含语句，蕴含语句的前提是正文字的合取，结论是单个正文字。
- 命题逻辑中，建立在限定子句上的**前向链接算法**：从知识库中的原子语句出发，在前向推理中应用**假言推理规则**，增加新的原子语句，直到不能进行任何推理。
- 如何应用于一阶限定子句，如何高效地实现。

一阶限定子句

- **一阶限定子句**中，假设变量是全称量化的，书写时省略全称量词
 - $\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 - $\text{King}(\text{John})$
 - $\text{Greedy}(y)$
- 有些知识库可写成限定子句的集合，但并非每个知识库都可写为限定子句的集合(单一正文字的限制过于严格)。

一阶限定子句

- 考虑以下问题：
 - 美国法律规定美国人贩卖武器给敌对国家是犯法的。美国的敌对国家Nono有一些导弹，所有这些导弹都是美国人West上校卖给他们的。
- 证明韦斯特是罪犯（criminal）。

定义常量、谓词，给出解释，写出逻辑语句

一阶限定子句

- 一阶限定子句表示这些事实
- “.....美国人贩卖武器给敌对国家是犯法的”
 - $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$ (1)
- “Nono.....有导弹。”
 - $\exists x Owns(Nono, x) \wedge Missile(x)$
 - 消去存在量词被转换成两个限定子句：
 - $Owns(Nono, M_1)$ (2)
 - $Missile(M_1)$ (3)
- 所有该国的导弹都购自West上校”
 - $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$ (4)

美国法律规定美国人贩卖武器给敌对国家是犯法的。美国的敌对国家Nono有一些导弹，所有这些导弹都是美国人West上校卖给他们的。

一阶限定子句

- “还需要知道导弹是武器：
 - $Missile(x) \Rightarrow Weapon(x)$ (5)
- 还需要知道美国的敌人被称为“敌对的”
 - $Enemy(x, America) \Rightarrow Hostile(x)$ (6)
- “West, 一个美国人……”
 - $American(West)$ (7)
- “Nono国, 美国的敌人……”
 - $Enemy(Nono, America)$ (8)
- 需要证明的结论: West是罪犯
 - $Criminal(West)$

美国法律规定美国人贩卖武器给敌对国家是犯法的。美国的敌对国家Nono有一些导弹, 所有这些导弹都是美国人West上校卖给他们的。

一阶限定子句

不包含函词，是**数据日志类**知识库。数据日志是一种受限于没有函词的一阶限定语句的语言。没有函词的推理更容易。

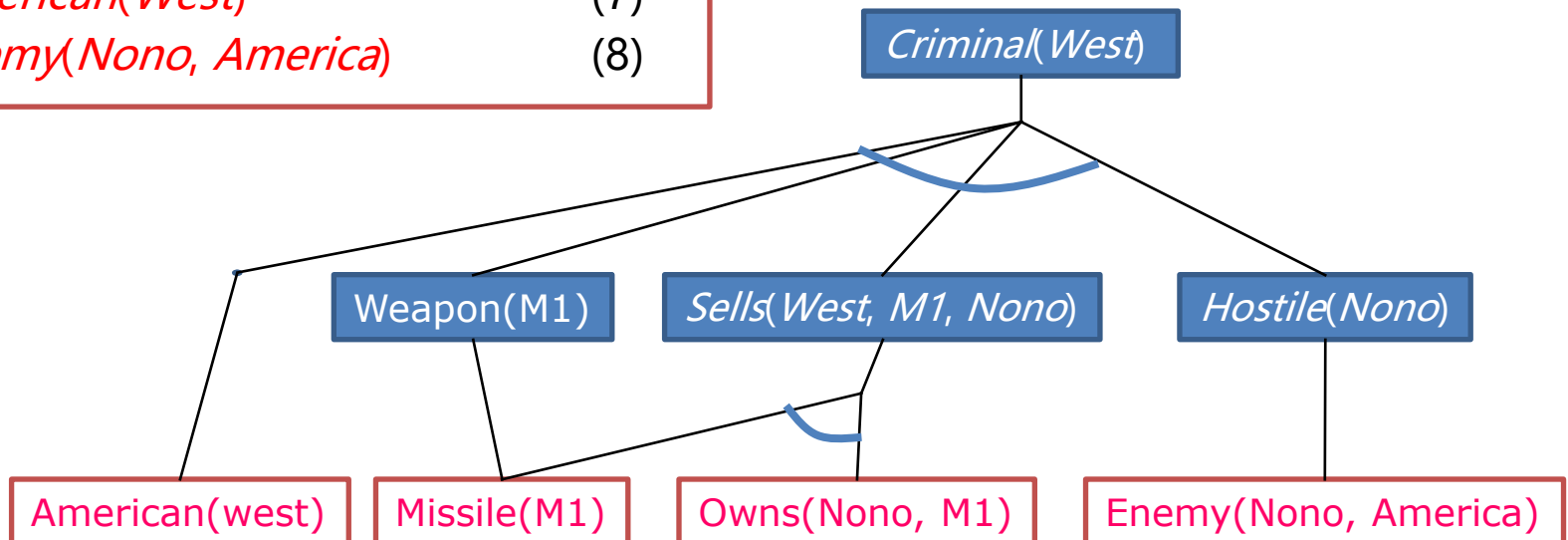
- $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$ (1)
- $Owns(Nono, M_1)$ (2)
- $Missile(M_1)$ (3)
- $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$ (4)
- $Missile(x) \Rightarrow Weapon(x)$ (5)
- $Enemy(x, America) \Rightarrow Hostile(x)$ (6)
- $American(West)$ (7)
- $Enemy(Nono, America)$ (8)

数据日志可以表示由关系数据库生成的语句类型。

前向链接算法

- $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$ (1)
- $Owns(Nono, M_1)$ (2)
- $Missile(M_1)$ (3)
- $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$ (4)
- $Missile(x) \Rightarrow Weapon(x)$ (5)
- $Enemy(x, America) \Rightarrow Hostile(x)$ (6)
- $American(West)$ (7)
- $Enemy(Nono, America)$ (8)

□ 使用前向链接算法进行证明



前向链接算法

function FOL-FC-ASK(KB, α) **returns** a substitution or false

inputs: KB, the knowledge base, a set of first-order definite clauses
 α , the query, an atomic sentence

local variables: new, the new sentences inferred on each iteration

repeat until new is empty

$new \leftarrow \{\}$

for each rule in KB **do**

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$

for each θ such that $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$
 for some $p'_1, \dots, p'_n$ in KB

$q' \leftarrow \text{SUBST}(\theta, q)$

if q' does not unify with some sentence already in KB or new **then**

add q' to new

$\phi \leftarrow \text{UNIFY}(q', \alpha)$

if ϕ is not fail **then return** ϕ

add new to KB

return false

考虑算法的效率



高效的前向链接



规则前提与已知事实进行匹配：合取排序

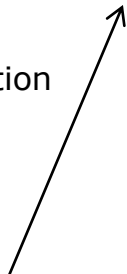
考虑规则与已知事实的匹配

□ 将规则前提与知识库中的已知事实匹配

找所有可能的合一
置换。这一过程称
为模式匹配，代价
昂贵

```

function FOL-FC-ASK(KB,a) returns a substitution or false
  inputs: KB, the knowledge base, a set of first-order definite clauses
           a, the query, an atomic sentence
  local variables: new, the new sentences inferred on each iteration
  repeat until new is empty
    new ← {}
    for each rule in KB do
      (p1 ∧ . . . ∧ pn ⇒ q) ← STANDARDIZE-VARIABLES(rule)
      for each θ such that SUBST(θ, p1 ∧ . . . ∧ pn) = SUBST(θ, p'1 ∧ . . . ∧ p'n)
        for some p'1, . . . , p'n in KB
          q' ← SUBST(θ, q)
          if q' does not unify with some sentence already in KB or new then
            add q' to new
            φ ← UNIFY(q', a)
            if φ is not fail then return φ
    add new to KB
  return false
  
```



考虑规则与已知事实的匹配

- 例如，考虑规则
 $Missile(x) \Rightarrow Weapon(x)$
- 需要找出所有能与 $Missile(x)$ 合一的事实；
- 对于已经建立了适当索引的知识库，这一过程对于每个事实都可以在常数时间内完成

- $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$ (1)
- $Owns(Nono, M_1)$ (2)
- $Missile(M_1)$ (3)
- $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$ (4)
- $Missile(x) \Rightarrow Weapon(x)$ (5)
- $Enemy(x, America) \Rightarrow Hostile(x)$ (6)
- $American(West)$ (7)
- $Enemy(Nono, America)$ (8)

合取排序

- 考虑规则 $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$
- **合取排序问题**：对规则前提的合取项进行排序，使总成本最小。
 - $Missile(x) \wedge Owns(Nono, x)$
 - $Owns(Nono, x) \wedge Missile(x)$

最少剩余值启发式

- 寻求最优排序是NP难题，但有好的启发式可用。例如，CSP的**最少剩余值 (MRV)** 启发式建议：
- 如果导弹数目少于Nono拥有的对象的数目，那么优先搜索导弹
 - $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$
- 否则，优先搜索Nono拥有的对象
 - $Owns(Nono, x) \wedge Missile(x) \Rightarrow Sells(West, x, Nono)$



避免重复匹配：
增量前向链接；逐步完成不完全匹配

应避免重复匹配

function FOL-FC-ASK(KB,α) **returns** a substitution or false

inputs: KB, the knowledge base, a set of first-order definite clauses

α, the query, an atomic sentence

local variables: new, the new sentences inferred on each iteration

repeat until new is empty

$new \leftarrow \{\}$

for each rule **in** KB **do**

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$

for each θ such that $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$

for some $p'_1, \dots, p'_n$ in KB

$q' \leftarrow \text{SUBST}(\theta, q)$

if q' does not unify with some sentence already in KB or new **then**

add q' to new

$\phi \leftarrow \text{UNIFY}(q', \alpha)$

if ϕ is not fail **then return** ϕ

add new to KB

return false

第二次迭代时 $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$ 与 $\text{Missile}(M_1)$ 再次匹配，而结论 $\text{Weapon}(M_1)$ 已知，所以什么都不会发生

如何避免再次匹配？

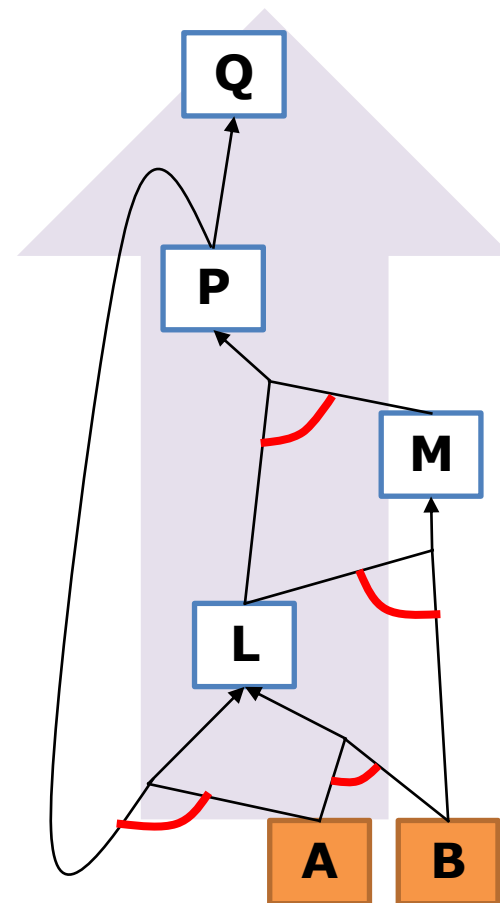
- ☐ $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$ (1)
- ☐ $\text{Owns}(\text{Nono}, M_1)$ (2)
- ☐ $\text{Missile}(M_1)$ (3)
- ☐ $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$ (4)
- ☐ $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$ (5)
- ☐ $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$ (6)
- ☐ $\text{American}(\text{West})$ (7)
- ☐ $\text{Enemy}(\text{Nono}, \text{America})$ (8)

增量前向链接

□ 增量前向链接：

必须使用前一次迭代中得到的事实进行本次迭代的推理得到新的事实

□ 不使用前一次迭代中得到的事实进行的推理可能在前一次迭代中就已经完成。



第1次迭代：从AB得到L

第2次迭代：从LB得到M

第3次迭代如果不使用M，只使用ABL，则只能得到L或M，它们已经在前面的迭代中获得。

逐步完成不完整匹配

- 第一次迭代时 $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$ 和事实 $American(West)$ 之间是**不完整匹配**。
- 简单前向链接算法中，该不完整匹配被**舍弃**并在后面迭代时重建。
- 改进：**保留**并**逐步完成不完整匹配**，而不是舍弃它们。

命题逻辑的前向连接使用_____实现了不完整匹配



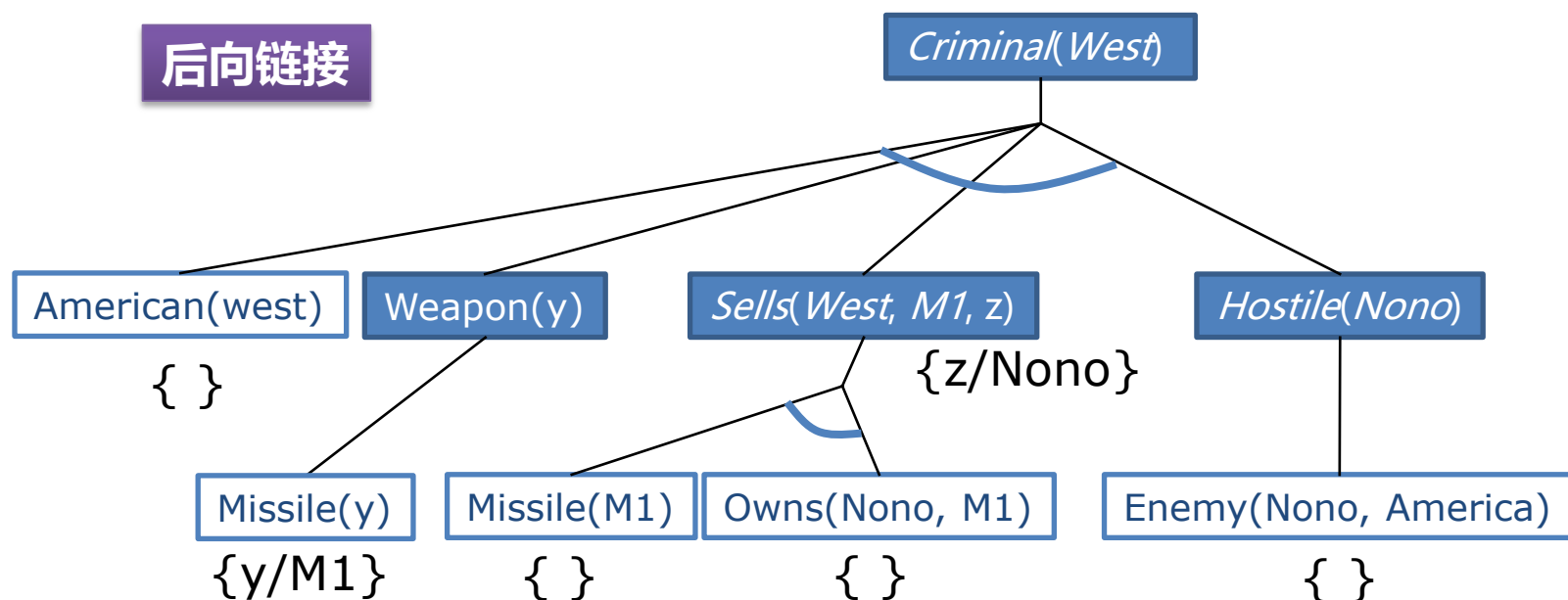
避免产生无关的事实

避免产生无关结论：魔法集

- 利用目标信息重写规则集，从而在前向推理过程中只考虑相关的变量绑定——魔法集。
- 若目标是 $Criminal(West)$,
- (1) 结论为 $Criminal(x)$ 的规则被重写以包含附加的合取子句，对 x 的取值进行约束：
 - $Magic(x) \wedge American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$
- (2) 事实 $Magic(West)$ 也被加入到KB中。这样，即使知识库中包括上百万美国人的事实，在前向推理过程中也只会考虑West。

后向链接算法

- 后向链接是一种深度优先搜索算法
- 一旦合取式中的某个子目标得以成功实现，它的置换要用于后续子目标。



一阶逻辑的合取范式CNF
一阶逻辑的归结推理规则
归结证明实例
归结策略

4 归结



合一代换

- $P \vee R, Q \vee \neg R$ can infer $P \vee Q$ using resolution in PL.
- can $P \vee R(x), Q \vee \neg R(y)$ infer $P \vee Q$?
- we need a **unifier** θ
where $SUBST(\theta, R(x)) = SUBST(\theta, R(y))$

Convert to CNF

- Essentially same as propositional logic
 1. Eliminate implications
 2. Move negation inwards
 3. Standardize variables
 4. Skolemize
 5. Drop universal quantifiers
 6. Distribute $\vee$ over $\wedge$
- Split Conjunctions into clauses

Resolution: FOL version

- in first-order logic:
- Resolution inference rule (for CNF)

$$\begin{array}{c}
 \ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n \\
 \hline
 \text{SUBST}(\theta, \ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n) \\
 \text{where UNIFY}(\ell_i, \neg m_j) = \theta.
 \end{array}$$

- Resolution Refutation(归结反驳)
 - Apply resolution steps to $\text{CNF}(\text{KB} \wedge \neg a)$ and infer contradiction (**empty clause**)

Examples

E1 1) $B(x)$

2) $\neg B(x) \vee C(x)$

3) $C(x)$

1)和2)

E2 1) $P(x) \vee Q(x)$

2) $\neg Q(f(y))$

3) $P(f(y))$

1)和2) $\{x/f(y)\}$

E3 1) $P(x, f(a)) \vee P(x, f(y)) \vee Q(y)$

2) $\neg P(z, f(a)) \vee \neg Q(z)$

3) $P(z, f(y)) \vee Q(y) \vee \neg Q(z)$

1)和2) $\{x/z\}$

4) $P(x, f(a)) \vee P(x, f(y)) \vee \neg P(y, f(a))$

1)和2) $\{z/y\}$

5) $Q(a) \vee \neg Q(z)$

1)和2) $\{x/z, y/a\}$

Resolution Steps

1. Convert problem into FOL sentence, including KB and query G .
2. Convert $\{KB, \neg G\}$ to a set of clauses.
3. Use resolution to infer new clauses from KB until producing a contradiction.

Example1

$$\square \quad (\forall x)(\forall y)\neg P(x, y), \\ (\forall x)(\forall y)(Q(x, y) \Rightarrow P(y, x)) \models \neg (\forall x) Q(x, a)$$

- 1) $\neg P(x, y)$ 前提
- 2) $\neg Q(x, y) \vee P(y, x)$ 前提
- 3) $Q(x, a)$ 结论的否定
- 4) $P(a, x)$ 2)和3) y/a
- 5) NIL 1)和4) $x/a, y/a$

单元优先

- **单元优先**：优先使用单文字语句（也称为**单元子句**）进行归结。要试图产生空子句，那么首先完成能够产生较短子句的推理。

将一个单元子句（如 P ）和任何其它子句（如 $\neg P \vee \neg Q \vee R$ ）归结时，总是生成一个比其它子句短的子句（ $\neg Q \vee R$ ）。

单元归结

- **单元归结**将归结限制为每次归结都必须包含单元子句。单元归结一般来说是不完备的，如果是Horn子句则是完备的。Horn子句的单元归结证明类似于前向链接

最佳优先搜索

- **最佳优先搜索**：用启发式函数度量每个子句的“权重”，权重小的优先执行。子句的权重应当与它的**规模和难度**相关联。单元子句权重轻；所以这种方法可以看作是**单元优先策略的一般化**。

OTTER定理证明器（Organized Techniques for Theorem-proving and Effective Research, McCune, 1992）使用了最佳优先搜索。

支撑集

- **支撑集**：坚持归结的**每一步都必须涉及至少一个特殊子句集**的元素，这个集合被称为**支撑集**。它首先确定一个称为**支撑集**的语句子集。归结式放到支撑集中。如果相对于整个知识库而言支撑集很小，搜索空间将会大幅度缩小。
- 完备性？

支撑集的选择

- 对支撑集的错误选择将会使得算法不完备。
如果我们选择的支撑集 S 使得剩余的语句是同时可满足的，那么支撑集的归结是完备的。
- 例如，假设原始知识库是相容的（所有语句可同时被满足），则**可用否定查询作为支撑集**，支撑集策略的一个额外优势是能够**生成目标引导的证明树**，这个证明树通常易于理解。

包容(subsumption)

- **包容**：包容法清除所有被知识库中的已有语句包容的语句。例如，如果 $P(x)$ 在KB中，则增加 $P(A)$ 毫无意义，增加 $P(A) \vee Q(B)$ 则更没有意义。包容帮助保持KB的小规模，这样才能帮助保持较小的搜索空间。

Thanks

NEXT: PLAN